

Agent 365 AI Teammates — Complete Step-by-Step Lab Guide

Version: Based on main branch • JTantra/a365-maf

Target audience: CPF Hackathon participants (no coding experience required)

Estimated time: 60–90 minutes

Azure Region: Southeast Asia (southeastasia)

What You Will Build

By the end of this lab, you will have a working **AI Teammate** that lives inside Microsoft Teams.

When a CPF staff member sends a message to the agent in Teams, it will:

- Understand the request (powered by Azure OpenAI)
- Look up relevant policy and vendor information
- Reply directly inside the Teams chat

✓	What Gets Created
✓	Agent identity in Microsoft 365 (like a user account, but for an AI)
✓	Agent blueprint registered in the tenant catalog
✓	Python backend deployed to Azure Container Apps
✓	Live Teams chat connected to the backend

Understanding the Toolbox — The Dev Container

Before running any commands, understand what you are working with.

Your CPF Laptop

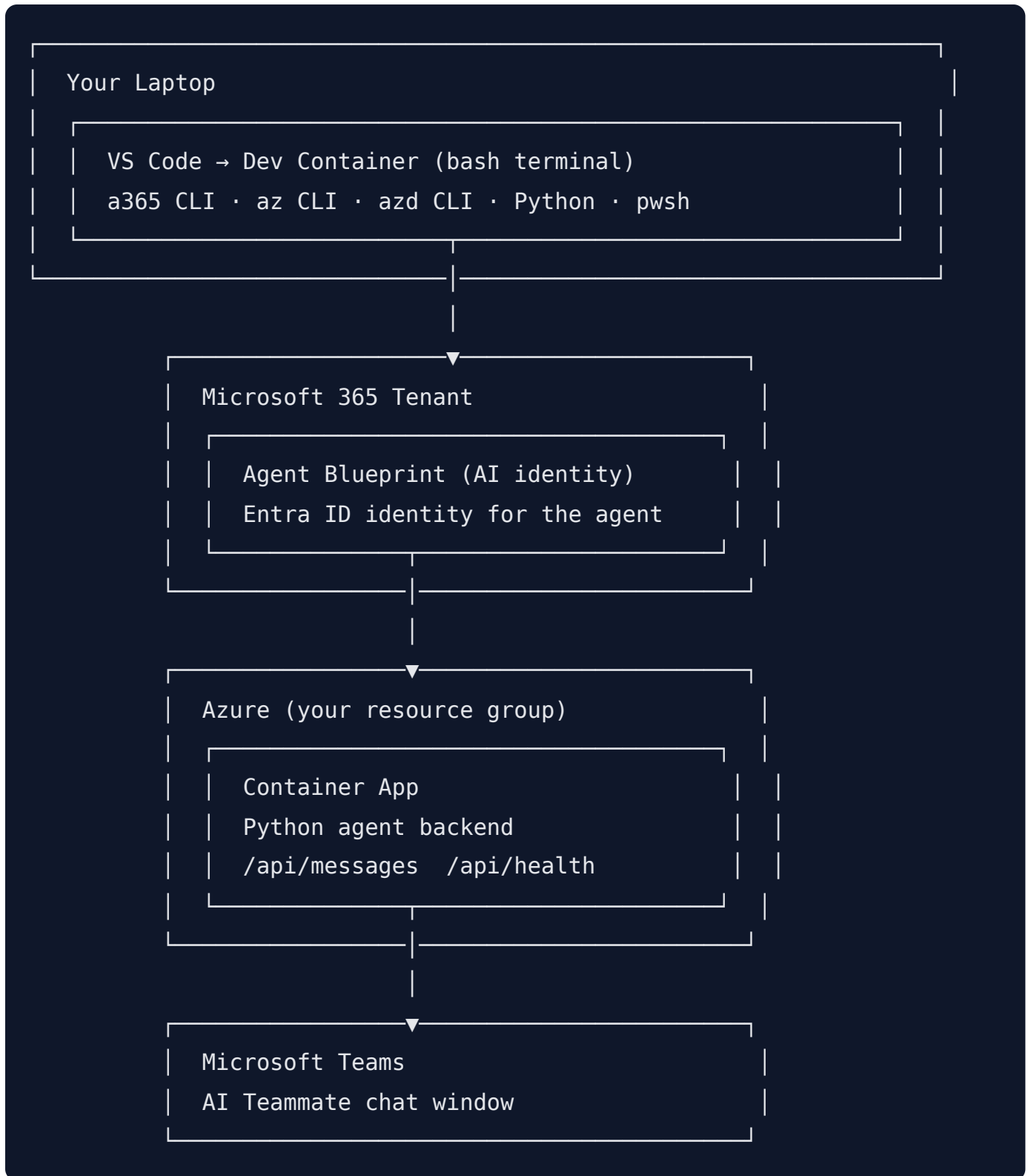
```
└─ Docker Desktop (runs a mini-Linux machine silently in the
background)
    └─ Dev Container (a pre-configured toolbox that lives inside
Docker)
        └─ a365 CLI ← creates the AI Teammate identity in Microsoft
365
            └─ az CLI ← talks to Azure (creates resources, checks
status)
                └─ azd CLI ← deploys the Python backend to Azure Container
Apps
                    └─ Python ← runs the agent code locally and packages it
for deployment
                        └─ pwsh ← PowerShell scripts for helper tasks
```

Why do we use a Dev Container?

- All 5 tools above are pre-installed — you do not install anything manually
- Every participant has identical tool versions — no "works on my machine" issues
- The container is disposable — if something breaks, rebuild it in 30 seconds
- All commands run inside Linux (bash) even though your laptop is Windows

Important: All commands in this guide must be run in the **bash terminal** inside the Dev Container — **not** in PowerShell, not in Windows Command Prompt, and not in any terminal outside VS Code.

Lab Architecture



Data flow when a user chats:

Teams user sends message

- Teams sends POST to /api/messages on Container App
- Python agent calls Azure OpenAI
- Azure OpenAI returns response
- Agent sends reply back to Teams
- User sees response in Teams chat

Before You Start — Facilitator Must Confirm

The **facilitator** completes these before the workshop. Participants do not do this.

#	What	Who
1	Tenant enrolled in Agent 365 preview	Facilitator
2	Participants have Agent ID Developer role in Entra ID	Facilitator
3	Participants have access to the shared Azure subscription	Facilitator
4	Agent 365 CLI app registration configured in tenant	Facilitator
5	Azure OpenAI resource and deployment available	Facilitator
6	Participants have the following values written down:	Facilitator
	• Azure Subscription ID	
	• Azure OpenAI endpoint URL	
	• Azure OpenAI deployment name	
	• Tenant ID	



CPF Hackathon: The facilitator pre-provides all values above. Use only what the facilitator gives you. Do NOT use your personal Azure subscription.

PHASE 1 — Agent Identity Setup

What this phase does: Creates the AI Teammate's identity in Microsoft 365.
Think of this as registering the agent as a "non-human staff member" in the tenant.

Where: All steps in this phase run in the **bash terminal inside the Dev Container**.

Step 1 — Open the Repository in VS Code Dev Container

What this step does

Opens the development environment. The Dev Container launches a mini-Linux machine with all tools pre-installed. You will do all lab work inside this container.

Prerequisites on your laptop (verify before starting)

Tool	How to check	Note
Visual Studio Code	Open it — if it opens, it's installed	https://code.visualstudio.com
Docker Desktop	VS Code will tell you if it's missing when you open the container	Must be installed, not manually launched
Dev Containers extension	VS Code → Extensions → search "Dev Containers" — look for  installed	Install from Extensions panel if missing

How to open the Dev Container

1. Open VS Code — that's all you need to do to start

You do **not** need to manually open Docker Desktop. The Dev Containers extension in VS Code handles Docker automatically. Docker Desktop must be **installed** on your laptop (the facilitator confirms this beforehand), but you never need to launch it yourself.

-  If you see an error like "Docker is not running" when VS Code tries to open the container → only then do you need to start Docker Desktop manually:
- Press **Windows key** → type Docker Desktop → press Enter → wait for the whale icon  in the taskbar to stop animating → then return to VS Code

2. Clone the repository (if not already done):

- Open VS Code
- Press `Ctrl+Shift+P` → type `Git: Clone`
- Enter: `https://github.com/JTantra/a365-maf`
- Choose a folder to save it (e.g. `C:\Users\<you>\repos\a365-maf`)

3. Open the a365-maf folder in VS Code:

- `File` → `Open Folder` → navigate to the `a365-maf` folder → click **Select Folder**
- ⚠ Must open the `a365-maf` folder itself — not a parent folder, not a child folder

4. Reopen in Container:

- VS Code will show a blue notification bottom-right: **"Reopen in Container"** — click it
- If no notification appears: press `F1` → type `Dev Containers: Reopen in Container` → Enter

5. Wait for the container to build:

- First build: **3–5 minutes** — watch the progress in the bottom-left corner of VS Code
- Subsequent opens: **30 seconds** — Docker reuses the cached image
- Do not close VS Code during the build

6. Open a bash terminal:

- `Terminal` → `New Terminal`
- Look at the **top-right of the terminal panel** — it must say **bash**
- If it says `pwsh` or `PowerShell` → click the `+` dropdown arrow → select **bash**

7. Navigate to the repo root and verify all tools are installed:

```
cd /workspaces/a365-maf
```

```
a365 --version
```

```
az version -o table
```

```
azd version
```

```
python --version
```

```
pwsh --version
```

What to expect

All 5 commands return version numbers without errors. Example:

```
a365 1.1.214
azd 1.25.6
az 2.87.0
Python 3.11.13
PowerShell 7.x.x
```

If a tool is missing (Optional)

```
# Reload the terminal profile
source ~/.bashrc

# If still missing, rerun the setup script
bash .devcontainer/setup-tools.sh

# Last resort – rebuild the container
# VS Code: F1 → "Dev Containers: Rebuild Container Without Cache"
```

 **CRLF issue on Windows:** If `setup-tools.sh` fails with a line ending error, run:

```
sed -i 's/\r//' .devcontainer/setup-tools.sh
bash .devcontainer/setup-tools.sh
```

Understanding the key files in this repository

Before running any commands, take 2 minutes to understand which files you will touch and which you must never edit:

File	Purpose	Do You Edit It?
<code>a365.config.example.json</code>	Sample agent config (tenant ID, agent name)	 Will let the CLI generate this later on

<code>ToolingManifest.json</code>	List of tools that the agents will require	✗ Don't manually edit, CLI to edit
<code>azure.yaml</code>	azd app definition and Docker build config	✗ Leave unchanged
<code>infra/main.bicep</code>	Azure infrastructure definition	✗ Leave unchanged
<code>.azure/<env>/.env</code>	Local azd environment state	✗ Auto-managed by azd

⚠ Verify secrets are excluded from git

Two files will be auto-generated later that contain secrets. Confirm they are in `.gitignore` now, before you proceed:

```
cat .gitignore | grep -E "generated|\.env"
```

You should see:

```
a365.generated.config.json
a365.generated.config.*.json
.env
.azure/
```

⚠ If you accidentally stage `a365.generated.config.json` at any point: `git reset HEAD a365.generated.config.json`

⚠ Never commit this file — it contains the agent's client secret.

Step 2 — Set Lab Variables

What this step does

Sets three shell variables that all later commands depend on. These must be set every time you open a new terminal.

Where to do this

In the **bash terminal** inside the Dev Container (the same terminal from Step 1).

Why not in PowerShell?

PowerShell uses a different syntax (`$env:AGENT_NAME = "..."`) and the a365/azd/az tools are only available on the bash PATH. Always use bash for this lab.

Commands — type these directly into the bash terminal

```
export AGENT_NAME="cpf-aor-team1"
export AZURE_LOCATION="southeastasia"
export AZURE_SUBSCRIPTION_ID="69ecbfc6-bfbb-4fa4-9933-a5ec21627a3f"
```

- ⚠ Replace `cpf-aor-team1` with your **team's unique name** (lowercase, hyphens only, no spaces).
- ⚠ Replace `<subscription-id-from-facilitator>` with the value the facilitator provides.
- ⚠ These variables disappear if you close the terminal or restart the container. Re-export them each new session.

Tip — set all three at once (easier for hackathon):

```
export AGENT_NAME="cpf-aor-team1" AZURE_LOCATION="southeastasia"
AZURE_SUBSCRIPTION_ID="<sub-id>"
```

How to verify

```
echo "Agent: $AGENT_NAME | Location: $AZURE_LOCATION | Sub:
$AZURE_SUBSCRIPTION_ID"
```

✅ Expected: All three values printed — no blanks.

Step 3 — Sign In to Azure

What this step does

Logs you in to Azure from inside the Dev Container. Two separate logins are required:

- `az login` — Azure CLI (for infrastructure commands)
- `azd auth login` — Azure Developer CLI (for deployment commands)

Both must use the **same account** provided by the facilitator.

Where to do this

In the **bash terminal** inside the Dev Container.

Commands — run these one at a time

3a. Azure CLI login:

```
az login --allow-no-subscriptions
```

A browser window opens → sign in with the **hackathon tenant account** the facilitator gave you

⚠ Do NOT sign in with your personal @microsoft.com account

3b. Set the correct subscription:

```
az account set --subscription "$AZURE_SUBSCRIPTION_ID"
```

3c. Azure Developer CLI login:

```
azd auth login
```

Browser opens again → sign in with the **same account** as 3a

How to verify

```
az account show --query "{user:user.name, tenantId:tenantId, subscriptionId:id}" -o json
```

✅ Expected output:

```
{
  "user": "yourname@hackathon-tenant.onmicrosoft.com",
  "tenantId": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "subscriptionId": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
}
```

⚠ Most common mistake: signing in with your @microsoft.com corporate account. The tenant ID must match what the facilitator provided. If it does not match, run `az logout` and repeat Step 3.

Step 4 — Validate Agent 365 Prerequisites

What this step does

Runs an automated check to confirm the tenant is ready for Agent 365. This talks to Microsoft 365 and checks that all backend services are provisioned.

Where to do this

In the **bash terminal** inside the Dev Container.

Command

```
a365 setup requirements
```

What to expect

- ✅ Green checkmarks next to each item = proceed
- ⚠ "Frontier preview warning" = acceptable, the facilitator has confirmed enrollment — continue
- ❌ Red failure = stop and call the facilitator — do not proceed until fixed

⚠ Known issue: If you see `Unrecognized command or argument '--agent-name'` — this is expected. Run the command without any flags, exactly as shown above.

Step 5 — Create the AI Teammate Blueprint

What this step does

This is the most important step. It creates the agent's **identity in Microsoft 365** — like registering a new (non-human) employee in the tenant. Specifically it:

- Creates an **Entra ID identity** for the AI Teammate (an app registration with a non-human account)
- Creates an **Agent 365 blueprint** — the template that defines the agent
- Configures **OAuth permissions** so the agent can read Teams messages
- Writes `a365.generated.config.json` with the blueprint ID and a secret

Where to do this

In the **bash terminal** inside the Dev Container.

Command

```
a365 setup all --aiteammate --agent-name "$AGENT_NAME"
```



Takes 2–5 minutes. Wait for it to complete fully.

What to expect in the terminal

```
Creating Entra application...  
Configuring permissions...  
Creating blueprint...  
Done.
```

How to verify — run this after the command completes

```
python3 -c "import json;  
print(json.load(open('a365.generated.config.json'))  
['agentBlueprintId'])"
```

✓ Expected: A GUID like xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx

✗ File not found: Re-run Step 5

How to verify in the browser (optional)

```
https://admin.microsoft.com  
→ Agents → Blueprints  
→ Your blueprint should appear in the list with status "Available"
```

⚠ "Blueprint already exists" message = fine. The command is idempotent. Continue.
⚠ NEVER commit `a365.generated.config.json` to git — it contains the agent's secret.

Step 6 — Publish the Manifest

What this step does

Packages the agent's metadata (name, description, icon) into a `manifest.zip` file. This zip file will later be uploaded to Microsoft Teams as the app package.

Where to do this

In the **bash terminal** inside the Dev Container.

Optional — update the display name (recommended for hackathon)

If you want your agent to show a friendly name in Teams, open the manifest file:

```
code manifest/manifest.json
```

Change only these two lines (leave everything else unchanged):

```
"short": "CPF AOR Team 1",  
"full": "CPF AOR Planning AI Teammate - Team 1"
```

💡 If you skip this, the agent will appear with a generic name in Teams. It still works — this is purely cosmetic.

⚠️ Do NOT change "manifestVersion": "devPreview" — this is required.

Command

```
a365 publish
```

How to verify

```
ls manifest/manifest.zip
```

✅ File exists → Phase 1 complete

PHASE 2 — Backend Deployment

What this phase does: Deploys the Python agent code to Azure. After Block 1, the agent has an identity but no brain. This phase creates the backend that actually processes messages and calls Azure OpenAI.

Where: All steps run in the **bash terminal** inside the Dev Container, **except** verification which can also be done in the Azure Portal.

Step 7 — Create the Azure Deployment Environment

What this step does

Tells `azd` (Azure Developer CLI) where to deploy: which subscription, which region, and what to name the resource group. This is like filling in an address form before sending a package.

Where to do this

In the **bash terminal** inside the Dev Container.

Commands — run in order

```
# Create a new named environment for your agent
azd env new "$AGENT_NAME"

# Tell azd where to deploy
azd env set AZURE_LOCATION "$AZURE_LOCATION"
azd env set AZURE_SUBSCRIPTION_ID "$AZURE_SUBSCRIPTION_ID"
azd env set AZURE_RESOURCE_GROUP "rg-$AGENT_NAME"

# Set the shared Azure OpenAI resource (get exact values from
facilitator)
azd env set AZURE_OPENAI_ACCOUNT_NAME "terenceaifoundry-resource"
azd env set AZURE_OPENAI_RESOURCE_GROUP "rg-admin-terenceaifoundry"
azd env set AZURE_OPENAI_ENDPOINT "https://terenceaifoundry-
resource.openai.azure.com"
azd env set AZURE_OPENAI_DEPLOYMENT "gpt-5.4"
azd env set AZURE_OPENAI_API_VERSION "preview"
```

 The Azure OpenAI values above are examples. Replace with exact values from your facilitator.

If the environment already exists (returning to a session):

```
azd env select "$AGENT_NAME"
```

How to verify

```
azd env get-values
```

 Expected: All variables listed with no blank values.

Step 8 — First Deployment: Create the Container App

What this step does

Deploys the Python agent backend to Azure. This single command:

- Builds a Docker image of the Python agent code
- Pushes it to Azure Container Registry
- Creates an Azure Container App to run it
- Creates a Log Analytics workspace for logs
- Gives you a public HTTPS URL your agent will respond at

Where to do this

In the **bash terminal** inside the Dev Container.

Command

```
azd up --no-prompt
```



5–8 minutes. This is normal — it is creating multiple Azure resources. Do not close the terminal.

What to expect in the terminal

```
Packaging services...
Provisioning Azure resources...
  (✓) Container Registry created
  (✓) Container App created
  (✓) Log Analytics workspace created
Deploying services...
  (✓) Python agent deployed

SUCCESS: Your application was provisioned and deployed.
```

How to get your public endpoint URL

```
FQDN=$(azd env get-values | grep AGENT_FQDN | cut -d= -f2 | tr -d '"')
echo "https://$FQDN/api/messages"
```


Save this URL — you will need it in Step 9.

How to verify the deployment is healthy

```
curl "https://$FQDN/api/health"
```

✓ Expected response:

```
{"status": "ok", "agent_type": "AgentFrameworkAgent",  
"agent_initialized": true}
```

✗ Connection refused or timeout → wait 2 minutes and retry (Container App may still be starting).

How to verify in the Azure Portal (optional)

```
portal.azure.com  
→ Resource Groups → rg-<your-agent-name>  
→ You should see: Container App, Container Registry, Log Analytics  
workspace
```

Step 9 — Connect the Blueprint to the Backend

What this step does

Right now the blueprint in Microsoft 365 (Step 5) and the Container App in Azure (Step 8) exist independently — they do not know about each other. This step links them by registering your Container App's URL as the messaging endpoint on the blueprint.

When a Teams user sends a message, Teams looks at the blueprint → finds the messaging endpoint → sends the message to that URL.

Where to do this

In the **bash terminal** inside the Dev Container.

Command

```
a365 setup blueprint --agent-name "$AGENT_NAME" --endpoint-only \  
--messaging-endpoint "https://$FQDN/api/messages"
```

How to verify in the terminal

✅ Expected: "Endpoint registered successfully" or similar confirmation message.

How to verify in the browser

```
https://admin.microsoft.com  
→ Agents → Blueprints → click your blueprint  
→ Messaging endpoint field should now show your Container App URL
```

Step 10 — Enable Authentication and Redeploy

What this step does

The Container App is running but does not yet know its own identity. When Teams sends a message, the Container App needs to verify it is genuinely coming from Microsoft Teams (and not from someone random posting to the URL).

This step:

1. Extracts the blueprint's credentials from `a365.generated.config.json`
2. Injects them into the Container App as environment variables
3. Redeploys so the Container App picks up the new credentials

Where to do this

In the **bash terminal** inside the Dev Container.

Commands — run in order

Extract the credentials from your generated config:

```
CLIENT_ID=$(python3 -c "import json;
print(json.load(open('a365.generated.config.json'))
['agentBlueprintId']))"
TENANT_ID=$(python3 -c "import json;
print(json.load(open('a365.config.json'))['tenantId']))"
SECRET=$(python3 -c "import json;
print(json.load(open('a365.generated.config.json'))
['agentBlueprintClientSecret']))"
```

Inject them into the azd environment:

```
azd env set BLUEPRINT_CLIENT_ID "$CLIENT_ID"
azd env set BLUEPRINT_TENANT_ID "$TENANT_ID"
azd env set BLUEPRINT_CLIENT_SECRET "$SECRET"

# Clear the secret from shell memory immediately (security best
practice)
unset SECRET
```

Redeploy the Container App with the new credentials:

```
azd up --no-prompt
```

 **3–4 minutes.** Faster than the first deployment — only the app is updated, not the infrastructure.

How to verify — run both tests

```
# Test 1: Health endpoint — must return 200
curl "https://$FQDN/api/health"

# Test 2: Messages endpoint — must return 401
curl -i "https://$FQDN/api/messages"
```

Result	Meaning	Action
--------	---------	--------

Health = 200 	Container App is running	Good
Messages = 401 	Authentication is working — rejects unauthenticated callers	This is correct
Messages = 200	Authentication not configured	Redo Step 10
Messages = 500	Application crashed	Check logs: <code>azd logs -</code> <code>-follow</code>



401 on the messages endpoint is the correct and expected result. It means the agent will only accept messages that come through the proper Microsoft Teams authentication flow — not random HTTP requests.

PHASE 3 — Publish to Teams and Test

What this phase does: Makes the agent visible and usable inside Microsoft Teams. After Phase 2, the backend is running and authenticated, but no Teams user can find or chat with the agent yet.

Where: Phase 3 uses both the **browser** (for admin steps) and **Microsoft Teams** (for testing), plus the **bash terminal** for watching live logs.

Step 11 — Upload the Manifest to M365 Admin Center

What this step does

Registers the agent in the Microsoft 365 tenant's app catalog. This makes the agent discoverable and installable in Microsoft Teams — similar to publishing an app to the Teams app store, but privately within your organisation.

Where to do this

In your browser — go to: <https://admin.microsoft.com>

Steps

1. In the left navigation, click **Agents → All agents**
2. Click "**Upload custom agent**" button (top right area)
3. When prompted, upload the file: `manifest/manifest.zip`
 - This file is in the `a365-maf` folder on your laptop
 - Path: `C:\Users\<you>\repos\a365-maf\manifest\manifest.zip`
4. In the upload wizard:
 - Review the agent name and description
 - Set **Activate scope** → select your team's pilot users (not "All users")
 - Click **Publish**

How to verify

Admin Center → Agents → All agents
→ Your agent name should appear with status "Published"

Step 12 — Create the Agent Instance in Teams and Chat

What this step does

Installs the agent for a specific user or group and creates the Teams chat interface. Once created, the agent appears as a contact in Teams that users can message directly.

Where to do this

In Microsoft Teams (desktop app or <https://teams.microsoft.com>)

Steps

1. In Teams, click **Apps** in the left sidebar
2. Search for your agent name (e.g. "CPF AOR Team 1")
3. Click the agent → click "**Create instance**"

- If you see **"Request instance"** instead → the facilitator needs to approve it first (see below)

4. Wait **2–5 minutes** for the agent account to appear in Teams

5. Find the agent in your Teams contacts → open a chat → type: **"hello"**

6.  The agent responds → lab complete

If approval is required (Request instance flow)

The facilitator approves from:

```
Admin Center → Agents → Requested agents → find your team's request →  
Approve
```

After approval, wait 2–5 minutes, then retry finding the agent in Teams Apps.

Step 13 — Watch Live Logs During Chat (Optional but Recommended)

What this step does

Streams live logs from the Container App while you chat in Teams. This lets you see every message that arrives, how the agent processes it, and confirms end-to-end connectivity.

Where to do this

In the **bash terminal** inside the Dev Container — run this while chatting in Teams in another window.

Setup — get resource details

```
RG=$(azd env get-values | awk -F=  
'/^AGENT_RESOURCE_GROUP=/{gsub("/","",$2); print $2}')
```

```
APP=$(azd env get-values | awk -F=  
'/^AGENT_CONTAINER_APP_NAME=/{gsub("/","",$2); print $2}')
```

Command — stream live logs

```
az containerapp logs show -g "$RG" -n "$APP" --tail 120 --follow
```

Now send a message in Teams. Within seconds you should see the log update.

What healthy logs look like

POST /api/messages HTTP/1.1" 200	← message received ✓
Validating agent and setting up context	← auth verified ✓
tenant_id=xxxxxxxx-xxxx-xxxx-xxxx...	← correct tenant ✓
Got it	← response sent to Teams ✓

Stop the log stream: `Ctrl+C`

Troubleshooting

Problem 1: Teams chat does not respond — no logs appearing

Diagnosis:

```
az containerapp logs show -g "$RG" -n "$APP" --tail 50 | grep  
'/api/messages'
```

If no `POST /api/messages` appears → Teams is not sending messages to your endpoint.

Fix — re-register the endpoint:

```
a365 setup blueprint --agent-name "$AGENT_NAME" --endpoint-only \  
--messaging-endpoint "https://$FQDN/api/messages"
```

Problem 2: Logs show 401 after authentication was set up

The `CLIENT_ID` on the Container App does not match the blueprint.

Diagnosis:

```
# What your blueprint expects
python3 -c "import json;
print(json.load(open('a365.generated.config.json'))
['agentBlueprintId'])"

# What the Container App has
az containerapp show -g "$RG" -n "$APP" \
  --query "properties.template.containers[0].env[?
name=='CLIENT_ID'].value" \
  -o tsv
```

If values differ → redo Step 10 and redeploy.

Problem 3: Agent starts but gives error responses

Diagnosis:

```
az containerapp logs show -g "$RG" -n "$APP" --tail 200 | grep -E
'Error|Exception|AADSTS|Failed|Invalid'
```

Error text	Meaning	Fix
AADSTS	Azure OpenAI permission missing	Ask facilitator
Invalid deployment	Wrong OpenAI deployment name	Check AZURE_OPENAI_DEPLOYMENT with facilitator
MCP consent	Tools not consented in Teams	Re-consent in Teams app settings

Problem 4: Variables are blank after reopening terminal


```
# Re-export all variables (do this every new terminal session)
export AGENT_NAME="cpf-aor-team1"
export AZURE_LOCATION="southeastasia"
export AZURE_SUBSCRIPTION_ID="69ecbfc6-bfbb-4fa4-9933-a5ec21627a3f"

# Re-select your azd environment
azd env select "$AGENT_NAME"

# Recover FQDN
FQDN=$(azd env get-values | grep AGENT_FQDN | cut -d= -f2 | tr -d '"')
```

Problem 5: Dev Container won't open

Symptom	Fix
"Docker not running" error	Start Docker Desktop, wait for green icon
VS Code opened wrong folder	File → Open Folder → select the a365-maf folder specifically
Container build fails	F1 → Dev Containers: Rebuild Container Without Cache
CRLF error in setup-tools.sh	sed -i 's/\r//' .devcontainer/setup-tools.sh then rebuild

Quick Reference — All Commands in Order

```
# — SETUP —————
cd /workspaces/a365-maf
cp a365.config.example.json a365.config.json
# Edit a365.config.json with your tenantId and clientAppId

# — VARIABLES —————
export AGENT_NAME="cpf-aor-team-changetouniquename"
export AZURE_LOCATION="southeastasia"
export AZURE_SUBSCRIPTION_ID="69ecbfc6-bfbb-4fa4-9933-a5ec21627a3f"

# — LOGIN —————
az login --allow-no-subscriptions
az account set --subscription "$AZURE_SUBSCRIPTION_ID"
azd auth login

# — IDENTITY —————
a365 setup requirements
a365 setup all --aiteammate --agent-name "$AGENT_NAME"
a365 publish

# — DEPLOYMENT —————
azd env new "$AGENT_NAME"
azd env set AZURE_LOCATION "$AZURE_LOCATION"
azd env set AZURE_SUBSCRIPTION_ID "$AZURE_SUBSCRIPTION_ID"
azd env set AZURE_RESOURCE_GROUP "rg-$AGENT_NAME"
azd env set AZURE_OPENAI_ENDPOINT "https://terenceaifoundry-
resource.openai.azure.com"
azd env set AZURE_OPENAI_DEPLOYMENT "gpt-4o"
azd env set AZURE_OPENAI_ACCOUNT_NAME "terenceaifoundry-resource"
azd env set AZURE_OPENAI_RESOURCE_GROUP "rg-admin-terenceaifoundry"
azd env set AZURE_OPENAI_API_VERSION "preview"
azd up --no-prompt

# — ENDPOINT REGISTRATION —————
FQDN=$(azd env get-values | grep AGENT_FQDN | cut -d= -f2 | tr -d '"')
a365 setup blueprint --agent-name "$AGENT_NAME" --endpoint-only \
  --messaging-endpoint "https://$FQDN/api/messages"
```

```
# — AUTH INJECT + REDEPLOY —————
CLIENT_ID=$(python3 -c "import json;
print(json.load(open('a365.generated.config.json'))
['agentBlueprintId']))")
TENANT_ID=$(python3 -c "import json;
print(json.load(open('a365.config.json'))['tenantId']))")
SECRET=$(python3 -c "import json;
print(json.load(open('a365.generated.config.json'))
['agentBlueprintClientSecret']))")
azd env set BLUEPRINT_CLIENT_ID "$CLIENT_ID"
azd env set BLUEPRINT_TENANT_ID "$TENANT_ID"
azd env set BLUEPRINT_CLIENT_SECRET "$SECRET"
unset SECRET
azd up --no-prompt
```

```
# — VERIFY —————
curl "https://$FQDN/api/health" # → 200 ✓
curl -i "https://$FQDN/api/messages" # → 401 ✓
```

```
# — LIVE LOGS —————
RG=$(azd env get-values | awk -F=
'/^AGENT_RESOURCE_GROUP=/{gsub("/","",$2); print $2}')
APP=$(azd env get-values | awk -F=
'/^AGENT_CONTAINER_APP_NAME=/{gsub("/","",$2); print $2}')
az containerapp logs show -g "$RG" -n "$APP" --tail 120 --follow
```

Tool Management

As part of governance and observability, Agent365 can also manage the tools that are available in your enterprise for you.

For MCPs already approved in tenant

Command	Purpose
---------	---------

<code>a365 setup permissions mcp --agent-name <name></code>	Re-grant MCP server OAuth2 grants on the existing blueprint depending on the <code>ToolingManifest.json</code> . Also <code>permissions bot / permissions custom / permissions copilotstudio</code> .
<code>a365 develop list-available</code>	List built-in MCP servers in the tenant catalog (what you can install).
<code>a365 develop list-configured</code>	List MCP servers currently wired into this agent (reads <code>ToolingManifest.json</code>).
<code>a365 develop add-mcp-servers <name> [<name>...]</code>	Add one or more built-in MCP servers to the agent (updates <code>ToolingManifest.json</code>).
<code>a365 develop remove-mcp-servers <name> [<name>...]</code>	Remove built-in MCP servers from the agent.

BYO MCPs

More information [here](#)

Command	Purpose
<code>a365 develop-mcp register-external-mcp-server</code>	Registers an external MCP server
<code>a365 develop-mcp evaluate</code>	Evaluates an mcp server

Cleanup — Only Run When Facilitator Confirms

 Do NOT run cleanup during the hackathon. Only run after the facilitator confirms the lab is finished.

```
# Delete all Azure resources for your agent  
azd down --purge
```

```
# Remove the blueprint from the tenant (optional)  
a365 cleanup blueprint --agent-name "$AGENT_NAME"
```
